

第二章

2.1 一个k臂赌博机问题

2.2 动作-价值方法

估计动作价值的方法

选择动作的方法

2.3 10臂赌博机问题

2.4 增量实现

2.5 跟踪一个非平稳过程

2.6 乐观初始值

2.7 基于置信度上界的动作选择

2.8 梯度赌博机算法

2.9 关联搜索

第三章 有限马尔可夫决策

3.1 “智能体-环境”交互接口

1) 交互过程长什么样?

2) 用 MDP 形式化：需要哪些东西

3) 马尔可夫性

3.2 目标与收益

3.3 回报和分幕

1) 分幕式任务

2) 持续性任务

3.4 将分幕式任务和持续性任务统一

1) 为什么要将两种方法统一?

2) 引入

3.5 策略和价值函数

1) 价值函数

2) 策略

3) 回溯图与贝尔曼方程

1. 基础关系：状态价值与动作价值的相互定义（一步关系）

2. 递归形态：贝尔曼方程（完整回溯）

4) v_{π} 是贝尔曼方程的唯一解

3.6 最优策略和最优价值函数

3.7 最优性和近似算法

第四章 动态规划

4.1策略评估

4.2策略改进定理

4.3策略迭代

4.4 价值迭代

4.5异步动态规划

4.6广义策略迭代（GPI）

4.7动态规划的效率

第五章 蒙特卡洛方法(MC方法)

5.1 蒙特卡洛预测

5.2 动作价值的蒙特卡洛估计

5.3 蒙特卡洛控制

什么是强化学习：

我自己：

强化学习就是通过和环境智能体交互时，设置一个奖励机制，这个奖励机制能让最大化总收益等价于实现目标，这样使得智能体学会一个策略来实现目标

OPENAI翁家翌：“只要存在反馈机制，你能把问题建模成一个环境，并且从环境中获得反馈，这个‘行动—反馈—更新’的闭环，本质上就是强化学习。”

第二章

强化学习与有监督的学习的对比

特性	强化学习	有监督学习
核心反馈机制	评估性反馈 这种反馈只表明刚才做的动作“好”还是“坏”（给多少分），但不告诉你是好在哪里，也不告诉你最好的动作是什么	指导性反馈 这种反馈直接给出“正确动作”是什么（标准答案），并且这个正确答案与你刚才采取的动作无关

特性	强化学习	有监督学习
数据来源	通过交互产生 智能体必须主动与环境交互，通过“试错”来产生数据和经验	外部提供 学习基于外部监督者提供的、既有的“带标注训练集”
是否依赖当前动作	依赖 反馈取决于你采取了什么动作。采取不同的动作，得到的反馈（收益）是不同的。	不依赖 正确的标签（Label）是客观存在的，无论你预测什么，标准答案都是同一个。
主要目标	最大化收益信号 寻找能让长期总收益最大化的策略。	推断与泛化 学习如何根据训练集，对新的、未见过的情境做出正确的分类或预测。
试探的需求	必须试探 因为不知道哪个动作最好，必须主动尝试不同的动作来发现高收益的选项。	无需试探 不需要通过试错来发现答案，而是通过修正误差来逼近老师给出的标准答案。
典型应用场景	交互式问题、未知领域（无法获得所有正确示例）、序列决策问题	模式分类、人工神经网络、系统辨识等有明确正确答案的任务

2.1 一个k臂赌博机问题

问题背景：一个有k个控制杆的赌博机，拉动不同的控制杆收益满足不同的分布

目的：在某一时间内最大化总收益的期望

为什么是总收益的期望而不是总收益：

因为我们无法把握收益，只能把握收益的期望。并且根据大数定律。我们最大化**期望**，是因为在概率环境中，**最大化期望是实现最大化未来实际总收益的唯一科学路径**。期望是因，实际收益是果。

名词解释：

- t时刻选择的动作记为 A_t ,相应的收益记为 R_t
- k个动作，每个动作在被选中时都有一个期望收益/平均收益，这个收益就叫做这个动作的**价值**,动作 a 的价值记为 $q_*(a)$ 定义为：
$$q_*(a) = E[R_t | A_t = a] \tag{1}$$

这个一般就是理论上的值，实际上我们也没法求这个的期望，只能对他进行一个估计。我们将动作 a 在 t 时刻的价值估计为 $Q_t(a)$

我们希望这估计值越接近 $q_*(a)$ 越好。任意时刻都会有价值估计最高的动作，而这个动作就被成为**贪心动作**。

- 利用（exploitation）：利用当前的知识得到更多的收益
- 试探：特意去选择那些非贪心的、甚至看起来较差的动作。收集更多信息，改善对这些动作价值的**估计**

如何平衡好“开发”和“试探”这很重要。

2.2 动作-价值方法

在 k 臂赌博机问题中，如何具体地去**估计每个动作的价值**，以及如何利用这些估计值来选择动作

估计动作价值的方法

采样平均方法估计动作价值：

$$Q_t(a) = \frac{t\text{时刻之前执行动作}a\text{获得的总收益}}{t\text{时刻之前执行动作}a\text{的总次数}} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{I}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{I}_{A_i=a}} \tag{2}$$

其中， $\mathbb{I}_{\text{predicate}}$ 表示随机变量，当 predicate 为真时其值为 1，反之为 0。当分母为 0 时，我们将 $Q_t(a)$ 定义为某个默认值，例如 $Q_t(a) = 0$ 。当分母趋向无穷大时，根据大数定律， $Q_t(a)$ 会收敛到 $q_*(a)$ 。

独立同分布的随机变量，样本均值会随着样本数的增大而逐渐趋于他们的数学期望

所以当 $N_t(a) \rightarrow \infty$ 时，

不过这个也只是针对平稳过程来讲的

选择动作的方法

1. 贪心方法

选择当前估计价值最高的那个动作

$$A_t \doteq \operatorname{argmax}_a Q_t(a) \quad (4)$$

2. ϵ -贪心方法

- 以概率 $1 - \epsilon$ 选择估计价值最高的那个动作
- 以概率 ϵ 随机选择一个动作

2.3 10臂赌博机问题

这一节是实验，要理解这个实验图像是这样的原因

2.4 增量实现

对于传统的求解方式：

$$Q_n = \frac{\sum_{i=1}^{n-1} R_i}{n-1} \quad (5)$$

这种传统的方式的弊端是需要存储历史数据

- 占内存
- 计算效率低

现在我们采用增量的方式来实现，推导如下

$$Q_{n+1} = \frac{\sum_{i=1}^n R_i}{n} \quad (6)$$

这个的意思是当动作a被选择n+1次之后动作a的价值的估计值。现在我们思考怎么能在 Q_{n+1} 的式子当中放上 Q_n

$$\begin{aligned} nQ_{n+1} &= (n-1)Q_n + R_n \\ Q_{n+1} &= (1 - \frac{1}{n})Q_n + \frac{1}{n}R_n \\ Q_{n+1} &= Q_n + \frac{1}{n}[R_n - Q_n] \end{aligned} \quad (7)$$

我们观察这个式子，它的形式我们可以理解为

$$\text{新的估计值} = \text{旧的估计值} + \text{步长} \times [\text{目标} - \text{旧的估计值}] \quad (8)$$

- “目标”= 这次更新时，我们希望当前估计去逼近的那个值。
- 这个里面 $\frac{1}{n}$ 我们可以理解为步长，更普适的式子是 $Q_{n+1} = Q_n + \alpha_t(a)(R_n - Q_n)$ ，步长可以随着时间进行变化。

在很多自然的问题(在线地、增量地、根据新反馈修正旧估计)当中我们发现都能推导成这个形式，于是人们意识到这个是一个很有用、很自然地一个框架

2.5 跟踪一个非平稳过程

- 平稳过程：环境不随时间变化——对应收敛，我希望估计 Q 最终稳定到一个固定值
- 非平稳过程：环境不随时间变化——对应跟踪，我希望估计 Q 能跟着真实值移动（哪怕永远有波动）

前边我们使用的步长 $\alpha = \frac{1}{n}$ 不适合于固定步长，因为学到后面你“变得很固执”：新数据几乎推不动 Q 。在非平稳环境里，希望**近期数据更重要**，否则你会一直被“很久以前”的回报拖着走，跟不上变化。那么我们就采用固定的步长。

$$Q_{n+1} = Q_n + \alpha[R_n - Q_n] \quad (9)$$

为什么固定步长之后能更加反应近期数据？原因是 Q_{n+1} 是由 R_i 和 Q_1 的加权平均值。并且越近期的数据权重越大。现在我们可以进行计算，通过迭代 $Q_{n+1} = (1 - \alpha)Q_n + \alpha R_n$

$$Q_{n+1} = \alpha R_n + (1 - \alpha)\alpha R_{n-1} + (1 - \alpha)^2 \alpha R_{n-2} + \cdots + (1 - \alpha)^{n-1} \alpha R_1 + (1 - \alpha)^n Q_1$$

$$Q_{n+1} = (1 - \alpha)^n Q_1 + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i \quad (10)$$

在平稳条件下，若对每个动作 a ，它被选择次数 $N_t(a) \rightarrow \infty$ ，并且该动作的步长序列 $\{\alpha_n(a)\}$ 满足

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty, \quad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty, \quad (11)$$

则 $Q(a)$ 的随机逼近更新在常见假设下可保证收敛（到相应的期望值/不动点）。

- 条件1：不能衰减得太慢，否则收敛不到呢就不动了
- 条件2：不能让噪声抖动太大，要不不好收敛。

2.6 乐观初始值

之前我们讲的 ϵ -**贪心策略** 相比于单纯的 **贪心策略** 更适合试探。现在我们不使用 ϵ -**贪心策略** 也可以进行试探。

假如我们在设置初始值 Q_1 的过程中将初始值设置地足够大，那么智能体在该策略下将会进行更多的**试探**

实验结果表明，采用乐观初始值的方法虽然在早期收敛较慢，但能够促使智能体进行更充分的探索，从而在长期内获得更优的策略性能。

2.7 基于置信度上界的动作选择

2.6节的乐观初始值探索主要发生在**初期**一旦估计“冷静下来”，探索就**自然消失**。对 **非平稳问题** 或 **长期不确定性** 并不理想，强烈依赖 Q_1 的设置于是作者在 2.7 节提出一个更“理性”的问题：**能不能根据“我们对某个动作有多不确定”，来决定是否再去探索它？**

本节就是在讲一个公式：

$$A_t \doteq \operatorname{argmax}_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right] \quad (12)$$

- 其中 c 是一个大于0的数，它控制着试探程度

2.8 梯度赌博机算法

梯度赌博机算法不再估计具体的价值，而是学习对每个动作的**数值化偏好**，它唯一的意义在于相对大小。

- **偏好函数** $H_t(a)$ ：算法为每个动作 a 维护一个偏好值 $H_t(a)$ 。这个值本身并没有具体的“收益”含义（比如它不是说这个动作能赚多少分），它唯一的意义在于**相对大小**
- 偏好越高，被选概率越大
- 动作选择的概率为 $\pi_t(a)$

下面是偏好函数和动作选择的概率之间的关系：

为了把“偏好”转化为具体的“选择概率”，算法使用了 **Softmax 分布**。

动作 a 在时刻 t 被选中的概率 $\pi_t(a)$ 计算公式为：

$$Pr\{A_t = a\} \doteq \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}} \doteq \pi_t(a) \quad (13)$$

这意味着：

- 所有动作的概率之和为 1。
- 偏好 $H_t(a)$ 越大，分子 $e^{H_t(a)}$ 越大，该动作被选中的概率 $\pi_t(a)$ 就越高。

本节的核心是偏好函数 $H_t(a)$ 的更新规则：

当选择了动作 A_t 并获得了收益 R_t 后

对于被选中的动作 A_t ：

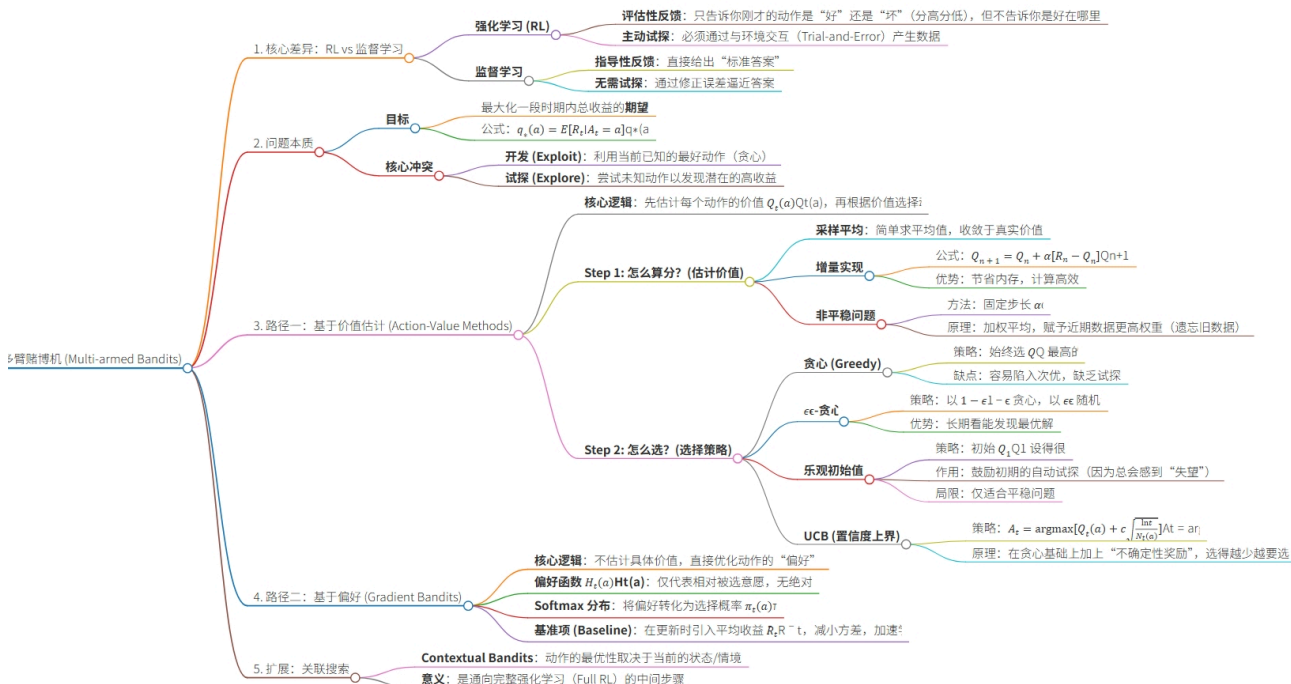
$$H_{t+1}(A_t) \doteq H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)) \quad (14)$$

对于未被选中的动作 a ($a \neq A_t$):

$$H_{t+1}(a) \doteq H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a) \quad (15)$$

2.9 关联搜索

这一节主要讲的就是最优动作会根据环境的不同而变换，所以针对这种情况改变相应的策略，但是具体怎么做本节还是没讲。



个人反思: 这个东西的最终目的是选择动作, 而选择动作有不同的策略(贪心, ϵ -贪心)

其中一个重要的选择动作的方法就是估计动作的价值。(当然还有一种是通过偏好, 那个和这个的“打法”不太一样)

第三章 有限马尔可夫决策

什么是有限MDP?

有限马尔可夫决策过程(Finite MDP)是强化学习问题在数学上的理想化形式, 它是对序列决策问题的一种形式化表达。

严格定义上, 它包含以下几个关键要素:

第一, 集合的有限性: 这也是它被称为“有限”的原因。指的是状态集合 $\mathcal{S}$ 、动作集合 $\mathcal{A}$ 以及收益集合 $\mathcal{R}$ 都只有有限个元素

第二, 环境的动态特性: 在这个框架下, 环境的变化是由一个四参数的概率函数 p 来完全描述的。即给定当前状态 s 和动作 a , 下一时刻转移到状态 s' 并获得收益 r 的概率是确定的, 记为 $p(s', r | s, a)$ 。因为集合是有限的, 所以这些随机变量具有定义明确的离散概率分布

思考下什么游戏是不能用这个式子来表示环境的特性的?

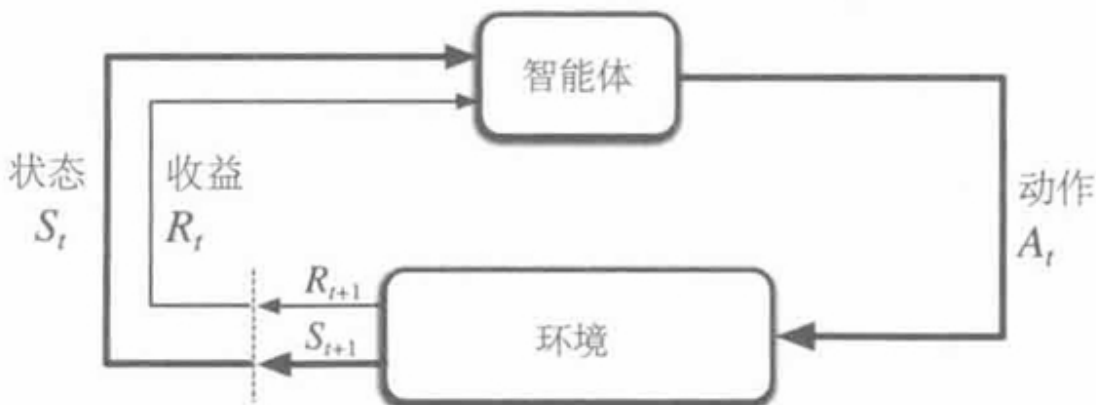
1. “我看不到全部”的游戏(破坏了全观测性)
2. “对手在不断进化”的游戏(破坏了静态环境假设)
3. “动作极其细腻”的游戏(破坏了有限性)

第三, 马尔可夫性: 这也是该过程的核心假设。即环境的未来状态和收益只取决于当前的状态和动作, 而与过去的历史完全无关

“老师，如果我在下象棋。**马尔可夫性**是指：我只需要看现在的棋盘残局（ S_t ），就能决定下一步怎么走，完全不需要知道咱们之前走了几百步是怎么走到这儿的。**环境特性**是指：具体的‘马走日，象走田’这些游戏规则，以及我吃掉对方将帅能得多少奖励的具体设定。”

最后，关于目标：在这个框架下，智能体的目标是最大化其收到的长期累积收益的概率期望值。”

3.1 “智能体-环境”交互接口



1) 交互过程长什么样？

本节用一条时间序列来描述交互：在时刻 t ，智能体观察到状态 S_t ，选择动作 A_t ，环境随后给出奖励 R_{t+1} 并转移到下一状态 S_{t+1} 。这就是强化学习反复循环的“感知→决策→反馈→新状态”。

2) 用 MDP 形式化：需要哪些东西

- **状态集合 S** : S_t 表示 t 时刻的状态
- **动作集合 $A(s)$** : A_t 表示在 t 时刻采取的动作
- **奖励集合 R** : R_{t+1} 表示在 t 时刻采取的动作后获得的奖励
- **环境动力学/转移规律**: 给出“在状态 s 做动作 a 之后，会到哪个 s' 并得到哪个奖励 r ”的概率。

3) 马尔可夫性

我们要说明什么是马尔可夫性：下一步（状态、收益）只和当前的状态和采取的动作有关

核心是定义一个联合概率：

$$p(s', r | s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a\} \quad (16)$$

它表达的就是：**未来一步只由当前状态和动作决定**，这就是“马尔可夫性”在这里的具体落地方式。

同时它必须满足归一化（对所有可能的 s', r 求和等于 1），书里也明确写了这一点。

另外，3.1 还把这个联合动力学拆成更常用的两个量：

- **状态转移概率** $p(s' | s, a) = \sum_r p(s', r | s, a)$
- **期望一步奖励** $r(s, a) = \mathbb{E}[R_{t+1} | s, a]$ （也能写成对 s', r 的加权求和）第三章

3.2 目标与收益

在强化学习中我们将目标设置为最大化总收益。（也就是用收益将目标信号化）

注意：收益的设置是很重要的，我们要保证**最大化总收益和实现目标这两件事是等价的**

这里面请注意：最大化总收益不应该是等价于实现目标的其中一个过程。也就是说我们不用传授智能体怎么实现目标。这样反而会导致最终目标实现地不够好

🧠 举个有意思的例子吧：

在人生这个游戏里，作为人的目标是最大化总收益。但是考上好的大学只是实现这个目标的一个路径上的一个阶段过程，但是有的人会将收益信号设置成总收益信号等价于考上好大学。最后导致实际的最大化总收益完成的并不好。

RL 的一切基石在于相信所有目标都能被量化为标量信号的累积和

3.3 回报和分幕

之前说的“最大化长期收益”，这个“长期收益” G_t 在数学上到底该怎么写？（本节主要是来解决这个问题。）

1) 分幕式任务

🧐 什么是分幕式任务

分幕式任务的定义

1. 定义

分幕式任务是指那些智能体和环境的交互可以被**自然地分解为一系列子序列**的任务。每一个这样的子序列就被称为一幕。

🧠 每一类分幕式任务在每一幕都会在有限步达到终结状态吗？

其实一个重要的判断依据就是有没有最终状态，比方说迷宫，虽然说在实际上可能存在永远也走不出来的情况，但是我们设置了最终状态，所以我们就把它算作分幕式任务

2. 核心特征：终结状态

这是分幕式任务最本质的数学特征：

- 每一幕都有一个特定的**最终时刻** T 。
- 每一幕都以一种**特殊的状态**结束，这个状态被称为**终结状态**。
- **重置机制**：当达到终结状态后，环境会被重置。下一幕会重新从某个标准的**起始状态**（或起始状态分布中的某个样本）开始。

3. 独立性

这是为了数学处理上的方便而做出的重要假设：

- 尽管每一幕结束的方式可能不同（例如：游戏赢了或输了），但**下一幕的开始状态与上一幕的结束方式完全无关**。
- 因此，我们可以认为这些幕是**独立重复**的交互过程。

4. 典型例子

书中列举了几个直观的例子来帮助定义：

- **一盘游戏**（如围棋、星际争霸）：有明确的输赢结局，结束后重新开局。
- **一次走迷宫的旅程**：走出迷宫即结束。

对于分幕式任务我们这样定义收益：

$$G_t \doteq R_{t+1} + R_{t+2} + \cdots + R_T \quad (17)$$

其中 T 是该幕结束的最终时刻。

2) 持续性任务

🧐 什么是持续性任务

任务没有自然的终点，智能体需要持续不断地进行交互，比如“调节室温的温控器”或“维持平衡的机器人”。

🧠 如果 $T = \infty$ ，上述简单求和公式可能会发散（趋于无穷大），导致无法比较策略优劣。

所以我们引入了折扣率 γ ($0 \leq \gamma \leq 1$)

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (18)$$

数学收敛性：只要收益序列 $\{R_k\}$ 是有界的，且 $\gamma < 1$ ，那么这个级数 G_t 就一定会收敛到一个有限值。这对于定义良构的优化问题至关重要。

γ 的物理意义：

- $\gamma \rightarrow 0$ ：“目光短浅”。智能体只关心眼前的即时收益 (R_{t+1})。
- $\gamma \rightarrow 1$ ：“高瞻远瞩”。未来的收益对当前的影响几乎不衰减。

递归性价值 G_t 满足这样的递归关系：

$$G_t \doteq R_{t+1} + \gamma G_{t+1}$$

当前的价值 = 即时收益 + 下一时刻价值的折扣值

(19)

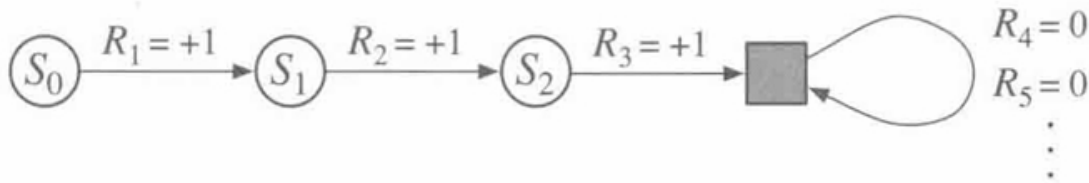
3.4 将分幕式任务和持续性任务统一

1) 为什么要将两种方法统一？

统一这两种任务不仅仅是为了“数学上的好看”，它在**理论推导**、**算法设计**和**代码实现**上都有着极其重要的实际意义。

2) 引入

- 首先是状态：原来我们用 S_t 表示 t 时刻的状态，但是现在我们用 $S_{t,i}$ 来表示在第 i 幕 t 时刻的状态。
- 把幕的结束当作一个特殊的吸收状态的入口。它只会转移到自己并且产生0收益。



(现在学得不是很明白，这一节)

形式上是把分幕式任务和持续性任务都统一成分幕式任务

3.5 策略和价值函数

1) 价值函数

价值函数 (Value Function)：用来评估智能体在给定状态（或给定状态-动作对）下，**若遵循某一特定策略**，未来预期能拿到多少回报。

在绝大多数强化学习应用中，我们无法计算真实值，必须依赖**估算 (Estimation)** 或**近似**

- 状态价值函数 $v_{\pi}(s)$

定义：从状态 s 开始，后续一直遵循策略 π 进行决策所获得的期望回报。

公式：

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t | S_t = s] \doteq \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s] \quad (20)$$

- 状态—动作价值函数 $q_{\pi}(s, a)$

定义：从状态 s 开始，执行动作 a ，之后所有可能的决策序列的期望回报。

公式：

$$q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] \doteq \mathbb{E}_{\pi}[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a] \quad (21)$$

2) 策略

策略定义了智能体的**行为方式**

它告诉智能体：“当你看到状态 s 时，你应该以多大的概率去选择动作 a ”

表达式记为： $\pi(a|s)$

这里的符号虽然像条件概率，但其实是一个**函数**。 $\pi: \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ 的函数。

我们需要拆解来看：

- **输入**：当前状态 s 。
- **输出**：选择动作 a 的概率。
- **含义**：如果在时刻 t 智能体处于状态 $S_t = s$ ，那么它采取动作 $A_t = a$ 的概率就是 $\pi(a|s)$ 。

关键性质：因为它定义了一个**概率分布**，所以在任何一个特定的状态 s 下，所有可能动作的概率之和必须等于 1：

$$\sum_{a \in \mathcal{A}(s)} \pi(a|s) = 1 \quad (22)$$

3) 回溯图与贝尔曼方程

构成要素：读图的“字母表”回溯图只有两种核心节点，非常容易辨认：

- 空心圆($\circ$)：代表一个 **状态 (State, s)**。
- 实心圆($\bullet$)：代表一个 **“状态-动作”二元组 (State-Action pair, (s, a))**。也就是在状态 s 下选择了动作 a 。

线条的含义：

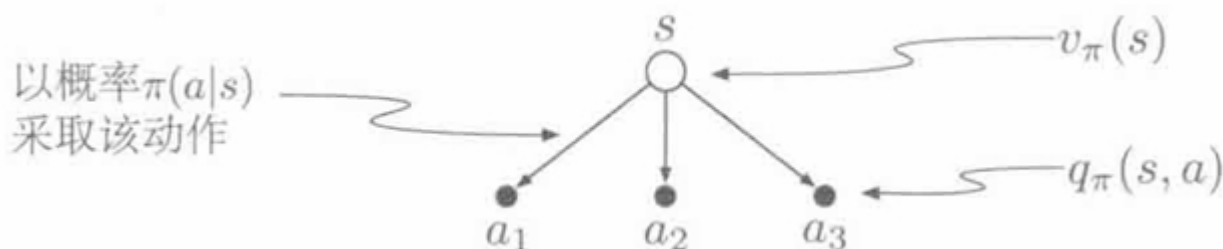
- 连接节点的线代表转换关系。
- 线旁边的符号（如 π 或 p ）代表转移的 **概率权重**。

下面是一些回溯图

1. 基础关系：状态价值与动作价值的相互定义（一步关系）

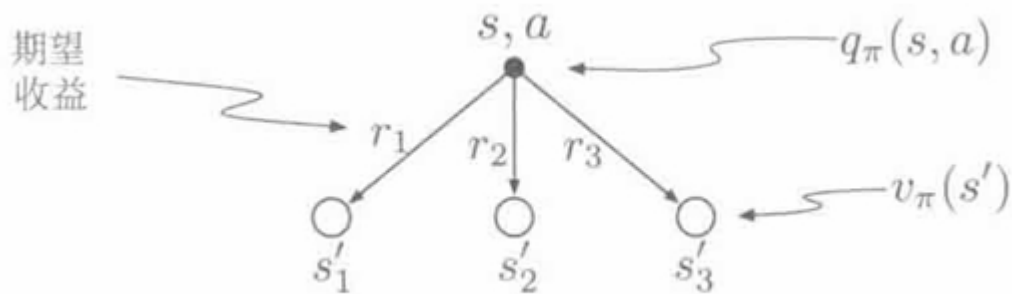
想要推导复杂的贝尔曼方程，必须先理解 v 和 q 是如何互相定义的。

- **利用 q 定义 v （状态价值是动作价值的加权平均）**



$$v(s) = \sum_a \pi(a|s) q_\pi(s, a) \quad (23)$$

- **利用 v 定义 q （动作价值是即时收益加打折后的未来状态价值）**

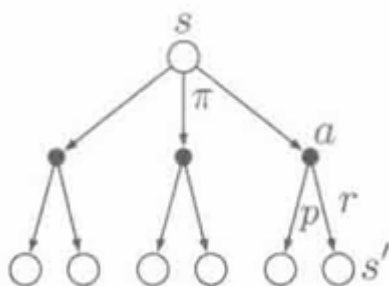


$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')] \quad (24)$$

2. 递归形态：贝尔曼方程（完整回溯）

将上述两个“积木”互相代入，就形成了价值函数的递归自我定义。

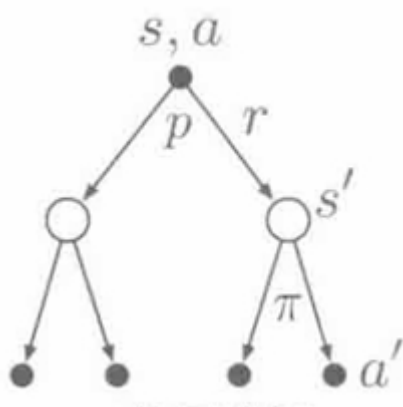
- v_{π} 的贝尔曼方程（状态价值的递归）



v_{π} 的回溯图

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')] \quad (25)$$

- q_{π} 的贝尔曼方程（动作价值的递归）



$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a')] \quad (26)$$

上面这些公式折扣率 γ 应该加在什么地方：

得到这一步的收益 r 之后后边的就都要加折扣率 γ

4) v_π 是贝尔曼方程的唯一解

先记住这个结论，实际的贝尔曼方程长什么样呢？

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_\pi(s')] \quad (27)$$

比方说有n个状态，那么我们就有n个方程，就有n个未知数

例：有两个状态 s_1, s_2

$$\begin{cases} v_1 = \underbrace{\left[\sum_a \pi(a|s_1) \sum_{s',r} p(s',r|s_1,a) \cdot r \right]}_{\text{常数}_A: \text{状态1的期望即时收益}} + \underbrace{\left[\gamma \sum_a \pi(a|s_1) \sum_r p(s_1,r|s_1,a) \right]}_{\text{系数}_B: \text{回到状态1的折扣概率}} \cdot v_1 + \underbrace{\left[\gamma \sum_a \pi(a|s_1) \sum_r p(s_2,r|s_1,a) \right]}_{\text{系数}_C: \text{转移到状态2的折扣概率}} \cdot v_2 \\ v_2 = \underbrace{\left[\sum_a \pi(a|s_2) \sum_{s',r} p(s',r|s_2,a) \cdot r \right]}_{\text{常数}_D: \text{状态2的期望即时收益}} + \underbrace{\left[\gamma \sum_a \pi(a|s_2) \sum_r p(s_1,r|s_2,a) \right]}_{\text{系数}_E: \text{转移到状态1的折扣概率}} \cdot v_1 + \underbrace{\left[\gamma \sum_a \pi(a|s_2) \sum_r p(s_2,r|s_2,a) \right]}_{\text{系数}_F: \text{留在状态2的折扣概率}} \cdot v_2 \end{cases} \quad (28)$$

3.6 最优策略和最优价值函数

一个策略比另一个策略更好的定义是什么？

如果策略 π 在所有状态下的期望回报都大于或等于策略 π' ，我们才说 π 优于或等于 π' （记作 $\pi \geq \pi'$ ）。

我们先在数学上承认这样一个事实（先不进行掌握）：在任何有限 MDP 中，一定存在一个策略*，在所有状态下，状态价值函数都是最优的，当然这样的策略不一定是唯一的但是我们仍然用 π_* 来表示所有这些最优策略，它们共享相同的 状态价值函数称之为最优状态价值函数。记为 v_* 。其定义为对于任意 $s \in S$

$$v_*(s) \doteq \max_{\pi} v_\pi(s) \quad (29)$$

最优策略也共享所有的最优动作价值函数，记为 q_* ，其定义为任意 $s \in S, a \in \mathcal{A}$

$$q_*(s, a) \doteq \max_{\pi} q_\pi(s, a) \quad (30)$$

因为 v_* 是某一个策略的状态价值函数所以肯定要满足贝尔曼方程

$$v_*(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_*(s')] = \sum_a \pi(a|s) q(s, a) \quad (31)$$

但是由于 π_* 是最优策略所以

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_{a \in \mathcal{A}(s)} p(s', r|s, a) (r + \gamma v_*(s')) \quad (32)$$

因此一个策略是最优策略充要于这个策略满足

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) = \max_{a \in \mathcal{A}(s)} p(s', r|s, a) (r + \gamma v_*(s')) \quad (33)$$

必要性前边讲的

充分性是根据这个东西的唯一性得到的

同理

$$q_*(s, a) = \sum_{s',r} p(s',r|s,a) [r + \gamma \max_{a'} q_*(s', a')] \quad (34)$$

知道最优策略的状态价值函数 v_* 或者 q_* 如何确定最优策略呢？（假设我们已经解出了贝尔曼最优方程）

- 知道 v_*

单步搜索后最好的动作就是最优动作

$$\pi^*(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')] \quad (35)$$

(收益+以后的最优策略下的价值最大，那当前选择的策略就是最优策略下应该选择的动作)

- 知道 q_*

$$\pi^*(s) = \arg \max_a q_*(s, a) \quad (36)$$

对于每个状态都存在一个或者多个动作，这个动作可以产生最大的价值.在一个策略中只有这样动作出现概率是非0的时候才可以

3.7 最优性和近似算法

我们应当明确几个现实问题

1. 现实世界中智能体几乎学不到最优策略（计算复杂度太大）原因是
 - 计算资源约束
 - 存储容量约束
2. 解决问题的两种表征方式

根据状态空间的规模，我们采取不同的对策：

- **表格型方法**：当状态集合足够小且有限时，可以用数组或表格来记录每个状态（或“状态-动作”二元组）的价值。
- **近似算法**：当状态太多、表格装不下时，必须采用紧凑的**参数化函数**（如神经网络）来表示价值函数

强化学习的优点就是可以进行在线运行,能在环境和智能体的交互过程中进行学习.但是我们要明确的这样的方法也有缺点，那就是在我们的训练过程中经常出现的状态我们能学习到比较好的策略但是不经常出现的状态我们遇到时候我们做出的策略可能没那么好（因为训练程度不够）

第四章 动态规划

什么是动态规划：DP是一类优化算法，在给定的MDP描述的环境模型中，可计算最优策略

对强化学习问题传统DP算法的缺点：

- 完备的环境模型（只是假设）
- 计算复杂度高

本章的大条件、背景：为有限MDP，并且 $p\{s', r|s, a\}$ 给出,当然言外之意也有这个问题是一个平稳的问题。

4.1 策略评估

评估一个策略的好坏，就要计算状态价值函数。这一节教我们如何用迭代法计算状态价值函数。

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_\pi(s')] \quad (37)$$

理论上，这是一个有着 $|s|$ 个未知数，以及 $|s|$ 个等式的线性方程组。所以只要 $\gamma < 1$ 或策略 π 在任何状态下都能保证最后终止，那就能解出唯一的 v_π ,但实际计算很繁琐。

使用迭代法来解决这类问题：近似状态价值函数序列， $v_0, v_1, \dots$ 从 s^+ 映射到 $\mathbb{R}$ (v_0 可任意选取，但终止状态必须为0)

更新规则：

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_k(s')] \quad (38)$$

显然 $v_k = v_\pi$ 是更新规则的一个不动点，在保证 v_π 存在的情况下，序列 $\{v_k\}$ 在 $k \rightarrow \infty$ 时会收敛到 v_π

2种更新方法：

- 旧的价值函数不变，新的价值函数一个一个算出来
- “就地”更新，每次直接用新的价值函数替换（这种算法的收敛速度更快）

4.2策略改进定理

策略改进定理, π 和 π' 是两个确定的状态, 对任意 $s \in S$, 只要

$$q_\pi(s, \pi'(s)) \geq v_{\pi'}(s) \quad (39)$$

那么一定有 $v_{\pi'}(s) \geq v_\pi(s)$

我们知道 q_π 和 v_π 可以有这样的表述形式:

$$\begin{aligned} v_\pi(s) &:= \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s] \\ q_\pi(s, a) &:= \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s, A_t = a] \end{aligned} \quad (40)$$

证明:

$$\begin{aligned} v_\pi(s) &\leq q_\pi(s, \pi'(s)) & (41) \\ &= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] & (42) \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] & (43) \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] & (44) \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}[R_{t+2} + \gamma v_\pi(S_{t+2}) \mid S_{t+1}] \mid S_t = s] & (45) \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s] & (46) \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \mid S_t = s] & (47) \\ &\quad \vdots & (48) \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s] & (49) \\ &= v_{\pi'}(s). & (50) \end{aligned}$$

策略改进:

$$\begin{aligned} \pi'(s) &\doteq \operatorname{argmax}_a q_\pi(s, a) \\ &= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \operatorname{argmax}_a \sum_{s', r} p(s', r \mid s, a)[r + \gamma v_\pi(s')], \end{aligned} \quad (51)$$

这样构造出来的策略他相比原策略一样, 甚至更好。上面针对的是确定性策略, 面对随机策略, 只要满足所有其他非最优动作的概率和为零即可。这种改进的策略不一定是最优策略, 但是一定比原策略要更优。

4.3策略迭代

策略 π 根据 v_π 产生了一个更好的策略 π' , 随后计算出 $v_{\pi'}$ 之后再得到更优的策略 π'' , 这样不断改进策略

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_* \quad (52)$$

这里 $\xrightarrow{\text{E}}$ 代表策略评估, 而 $\xrightarrow{\text{I}}$ 表示策略改进。

有限MDP只有有限种策略, 所以在有限次迭代之后一定能收敛到最优策略与最低价值函数。

4.4 价值迭代

策略迭代的缺点是每次都要进行策略评估 (要多次迭代, 直到 v_π 收敛)

$$\begin{aligned} \text{策略迭代} &= \text{在一次遍历后就停止策略评估的策略迭代} \\ &= \text{截断策略评估} + \text{策略改进} \end{aligned} \quad (53)$$

价值迭代更新公式:

$$v_{k+1}(s) \doteq \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] = \max_a \sum_{s', r} p(s', r \mid s, a)[r + \gamma v_k(s')] \quad (54)$$

证明: 从第 k 次迭代的价值函数 v_k 开始:

策略改进:

$$\pi_{k+1} = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')] \quad (55)$$

策略评估：接下来使用新策略（以100%选择arg max的动作）来执行一次评估迭代

$$\begin{aligned} v_{k+1}(s) &= \sum_a \pi_{k+1}(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')] \\ &= \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')] \end{aligned} \quad (56)$$

可以证明对于任意的 v_0 ，只要 v_* 存在，那么序列 $\{v_k\}$ 都可以收敛到 v_*

从另一个角度来看价值迭代就是借助贝尔曼最优方程进行迭代

价值迭代和策略迭代的区别：之前评估和改进是分开进行的，但是这个价值迭代里面max同时起到了评估和改进的作用。

4.5 异步动态规划

💡异步动态规划和传统DP的问题有什么区别：

传统DP的问题：在这一节之前介绍的策略迭代和价值迭代，都存在一个“主要缺点”：它们是**同步的**。这就导致单次遍历十分昂贵。异步DP算法是一类就地迭代的DP算法。**“异步”的含义**：异步DP算法不以系统遍历状态集的形式来组织算法。

💡异步动态规划有什么意义：

避免遍历并不意味着我们可以减少计算量。只意味着一个算法在改进策略前，不需要陷入漫长而无望的遍历。我们可以试着选择一些特定状态来更新，从而加快算法速度。我们也可以试着调整更新地顺序，让价值信息能更有效地在状态间传播。一些状态不用频繁更新价值函数，甚至一些状态和最优行为无关，我们完全可以跳过。

异步动态规划使计算和实时交互变得更容易，我们可以在一个智能体实际在MDP中进行真实交互的同时，执行DP算法。

4.6 广义策略迭代（GPI）

用广义策略迭代一词来指代让策略评估和策略改进相互作用的一般思路，与这两个流程的粒度和其它细节无关。几乎所有的强化学习方法都可以被描述为GPI。

在这里，“粒度”指的是**策略评估和策略改进这两个流程穿插的“深度”或“精细程度”**

简单来说，它回答了这样一个问题：“我们（在切换到另一个流程之前）在一个流程上要花多少功夫？”

我们来看几个例子：

1. 大粒度

- **例子**：标准的策略迭代
- **表现**：它的粒度非常“粗”或“大”。
 - **评估流程**：你必须执行完整的策略评估，即反复迭代，直到价值函数 V 完全收敛
 - **改进流程**：然后，你再执行一次完整的策略改进
- 这两个流程严格交替，每一步都“很大很完整”

2. 细粒度

- **例子**：价值迭代
- **表现**：它的粒度就“细”得多。
 - **评估流程**：它不等评估收敛，而是把评估“截断”到只进行一次迭代（即只遍历一次所有状态）
 - **改进流程**：然后它立刻就进行策略改进（通过 max 操作实现）
- 它把评估和改进在每一次遍历中都结合了起来

3. 极细粒度 (Very Fine Granularity)

- **例子**：异步动态规划 (Asynchronous DP)
- **表现**：粒度可以细到极致。
 - **评估流程**：可能只更新一个状态的价值。
 - **改进流程**：然后马上就对这个状态的策略进行改进

4.7 动态规划的效率

🔥 DP算法的效率是多项式级别的：

n 是算法中的状态数量， k 是算法中的动作数量。那么DP方法的计算操作次数是 n 和 k 的某个多项式函数（例如 n^2k, n^3 等）这种操作次数可以认为这个算法是高效的。

🔥 维度灾难

状态集合太大（维度灾难）是**问题本身的困难**，而不是 DP 作为一种解法所带来的困难。**DP的优势**：事实上，相比于直接搜索和线性规划，DP **更加适合**解决大规模状态空间的问题。

🔥 实际应用效率

最后，本节讨论了DP在实际中的应用情况：

- **实际性能：**
 - 在当今的计算机上，DP 算法可以解决拥有**数百万个状态**的 MDP。
 - 在策略迭代和价值迭代中，没有定论哪种更优。
 - 在实际应用中，这些方法的收敛速度通常**比它们理论上的最坏情况要快**，特别是当设置了好的初始价值函数或策略时。
- **异步DP的角色：**
 - 对于那些状态空间极其巨大的问题，我们通常使用**异步DP算法**。
 - 原因是，传统的“同步”DP算法（如标准价值迭代）要求在一次遍历中计算和存储**每一个状态**的值，这在存储和计算上是“不切实际的”。
 - 异步DP之所以可行，是因为在很多问题中，**只有相对较少的状态会沿着最优解的轨迹出现**。
 - 异步算法可以利用这一点（正如 4.5 节所讨论的），聚焦于这些相关的状态，因此可能比同步方法**更快地**找到最优策略。

第五章 蒙特卡洛方法(MC方法)

本章大背景：

- 没有完备的环境知识 $p(s', r|s, a)$ 未知
- 分幕式任务（否则无法终止，如果一个任务没有终点，即非分幕式任务，那么“回报”的计算将是无限的，你就永远等不到一个最终的“总收益”值，也就无法取平均。）
- 学习基于完整的回报（无法做到“在线学习”）

MC方法它与DP算法的区别是：它不假设拥有完备的环境知识，即状态转移的概率 $p(s', r|s, a)$ ，因为它可以从与真实的环境交互中学习经验，而不需要关于动态环境的先验知识。

5.1 蒙特卡洛预测

本节讲的内容是：在给定一个策略后，如何估算出这个策略下每一个状态 s 的价值函数 $V_{\pi}(s)$

- 首次访问型 MC 算法 (First-visit MC)：在估计状态 s 的价值时，只使用智能体在每一幕中第一次访问到状态 s 之后的回报。然后，它对所有这些“首次访问”的回报取平均值。
- 每次访问型 MC 算法 (Every-visit MC)：在估计状态 s 的价值时，会使用智能体在每一幕中每一次（包括第一次、第二次...）访问到状态 s 之后的所有回报。然后，它对这所有访问的回报取平均值。

当 s 的访问次数趋向无穷的时候，首次访问型MC和每次访问型MC都会收敛到 $v_{\pi}(s)$

对于首次访问来说，每次首次访问是相互独立的，这样就满足大数定律。但是对于每次访问来说，每次访问不是相互独立的。

蒙特卡洛算法的一个重要的事实是：它对于每个状态的估计是独立的。它对于每个状态的估计完全不依赖于对其他状态的估计。换言之它与DP算法不同的是没有用到前边的**自举思想**。

例子：

例5.1 [虚拟二十一点 - 练习21点纸牌游戏](#)（可以点击网址实际操作更好地理解二十一点的规则）

例5.2 肥皂泡这个例子是为了帮我们形象地体会MC算法和DP算法的计算方法

5.2 动作价值的蒙特卡洛估计

核心目标是利用蒙特卡洛（MC）方法来估计**动作价值函数** $q_{\pi}(s, a)$

评估计算 $q_{\pi}(s, a)$ 的方法

- 每次访问型：每次访问型 MC 算法使用某一策略 π 下，所有访问过“状态-动作”二元组 (s, a) 之后得到的**所有回报**的平均值，作为 $q_{\pi}(s, a)$ 的近似
- 首次访问型：首次访问型 MC 算法将某一策略 π 下，**每幕中第一次**在这个状态 s 下采取动作 a 之后得到的回报的平均值，作为 $q_{\pi}(s, a)$ 的近似

🙄 缺点：这个里面的复杂之处在于一些“状态-动作”二元组可能永远也不会访问到。

💡 克服上面问题的方法（一些“状态-动作”二元组可能永远也不会访问到）

- 试探性出发：将指定的“**状态-动作**”二元组作为起点开始一幕采样。这样可以保证，在采样的幕数趋于无穷大时，每一个“状态-动作”二元组都会被访问到**无数次**。
- 只考虑那些在每个状态下所有动作都有可能被选中的随机策略。（有的策略咱们干脆就不评估）

5.3 蒙特卡洛控制

把评估和改进结合起来，寻找**最优策略**。

首先我们先回顾一下前边的DP章节中介绍的广义策略迭代。

📦 根据前边DP方法中的**策略迭代**直接套用——经典策略迭代算法的蒙特卡洛版本

假设：

假设一：无限多幕样本。它要求在每一次改进策略之前，必须先进行无限次采样，把价值函数 q_{π} 算得完全精确

假设二：试探性出发。它要求必须保证所有的动作都能被选为起点，以确保所有状态都能被访问到

从策略 π_0 开始交替进行策略评估和策略改进，最终得到最优策略和动作价值函数

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \pi_2 \rightarrow \cdots \xrightarrow{I} \pi_* \xrightarrow{E} q_* \quad (57)$$

1. **策略评估**：这一步我们在5.2节已经学过了。给定一个策略 π ，我们通过大量的采样经验，让价值函数 Q 逼近真实的 q_{π}
2. **策略改进**：这一步很简单。一旦我估算出了每个动作的价值 $Q(s, a)$ ，我就变得“贪心”一点。对于每个状态 s ，我直接选择那个 Q 值最大的动作

公式如下：

$$\pi(s) \doteq \arg \max_a q(s, a) \quad (58)$$

根据策略改进定理，这样生成的贪心策略 π_{k+1} 至少会和原来的 π_k 一样好，甚至更好。这样“评估-改进-再评估-再改进”一直循环下去，最终就会收敛到最优策略 π_* 和最优价值函数 q_*

🙄 经典策略迭代算法的蒙特卡洛版本的缺点

理论上，要让这个循环完美运作，我们需要在每一次“改进”之前，进行“无限多”次采样来精确地完成“评估”。

但在实际操作中，这是不可能的。我们不可能为了改一次策略，就把游戏玩到天荒地老。有两种解决方法：

1. **近似法**：设定一个误差范围，只要估算得差不多就行。

2. **逐幕更新**：这是本节的重点。我们**不再等待**。

我们采用一种极限思想：**每玩完一幕，我就立刻进行评估，然后立刻改进策略**。这就引出了本节最重要的算法——**Monte Carlo ES**。